BrandStore — Launch Report

Live URL: TO FILL (after deploying, see `docs/RUNBOOK.md`) | **Release:** v1.0.0 (see `CHANGELOG.md`) | **Date:** 2026-11-12 | **Author:** Ahmed

1. What was built

BrandStore is a small online-shop front page: a landing page (hero offer, four categories, a filterable list of three products, a scrolling strip of six launch offers, two customer quotes and an email sign-up), a side-by-side product comparison page, and a friendly "page not found" page. It is plain HTML, CSS and JavaScript with no server, so it loads quickly and can be hosted for free. The sign-up form checks the address and responds on screen, but nothing is sent or stored; there is no checkout, login or database in this version.

2. Routes (what a visitor can open)

Route	What it is
`/` (`index.html`)	Landing page; sections `#categories`, `#featured`, `#deals`, `#testimonials`, `#newsletter`
`/comparison.html`	Product comparison table
any unknown address	`404.html` (Netlify)
`/robots.txt`, `/sitemap.xml`	Files for search engines
Backend / API	none (static site; sign-up is simulated)

3. Key decisions and deviations

- **No frameworks or build step.** Anyone can open the folder and edit it; the trade-off is that the product list lives in a JavaScript file rather than a real database.
- **Accessibility first:** a "skip to main content" link, labelled page regions, every control reachable by keyboard with a visible focus ring, errors read out by screen readers, and a calm motion setting for people who ask for reduced animation.
- **Search and sharing:** every page has its own title and description, share-preview tags and a 1200×630 preview image; `robots.txt`, a sitemap and a custom 404 page are included.
- **Security headers** (Netlify only): the page may only load its own scripts, styles and images, and cannot be embedded in other sites.
- **Deviation 1:** the web address is not known until deployment, so share-preview and canonical links hold a placeholder until `node scripts/set-site-url.js <url>` is run. A check (`node qa/preflight.js --release`) refuses to pass while the placeholder remains.
- **Deviation 2:** the 404 page only displays correctly at the site root; a mistyped deep address such as `/a/b` loads the page but its images and styles use relative paths. Fix next: make asset paths root-relative.

4. QA matrix (Chrome, emulated screens)

Same script on every row (`node qa/matrix.js`): loads without console errors, no sideways scrolling, three products, quotes, menu or links, filter, deals buttons, newsletter error and success, skip link, comparison table, 404 page.

Browser / screen	Result	Notes
Chrome 153, desktop 1440×900	PASS	measured
Chrome 153, laptop 1024×768	PASS	measured
Chrome 153, tablet 768×1024 (iPad Mini profile)	PASS	measured, emulated
Chrome 153, phone 375×667 (iPhone SE profile)	PASS	measured, emulated
Chrome 153, phone 390×844 (iPhone 13 profile)	PASS	measured, emulated
Chrome 153, phone 412×915 (Pixel 7 profile)	PASS	measured, emulated
Firefox (desktop)	TO FILL	not run in the build environment
Safari or Edge	TO FILL	not run in the build environment
A real phone	TO FILL	the phone rows above are emulations

5. Lighthouse and axe results

Tool	Page	Result
Lighthouse (mobile): Performance / Accessibility / Best Practices / SEO	landing	TO FILL
Lighthouse (mobile): same four scores	comparison	TO FILL
axe DevTools: Critical / Serious / Moderate / Minor	landing	TO FILL
axe DevTools	comparison	TO FILL

Tool	Page	Result
Screen reader (NVDA / VoiceOver / TalkBack) on the landing page	landing	TO FILL (one line per step in the runbook)

Offline stand-in, clearly not the same tool: our own audit `node qa/preflight.js` mirrors the axe and Lighthouse rules for language, titles, landmarks, headings, names, alt text, contrast-related checks, skip link, reflow and SEO tags. It passes 148 of 148 checks on all three pages (150 of 150 after the real URL is set). The earlier contrast audit measured the lowest text contrast at 5.02:1 (the standard asks for 4.5:1). Success needs Lighthouse Accessibility and SEO of at least 90 and no Critical axe issue; those numbers must come from the live site.

6. A real bug found and fixed during QA

Symptom: with the browser's text size enlarged to 200%, the last links in the top menu were cut off at the right edge on desktop widths, so a visitor could not read or click them. **Found by:** the audit that enlarges text. **Cause:** the menu was a single non-wrapping row. **Fix:** let the header and menu wrap onto a second line. **Check:** the screenshot at 200% now shows every link, and the layout, keyboard and preflight checks still pass. (Earlier, in Task 4, a different bug was found and fixed: an old error message could reappear a moment after a successful sign-up. Details are in README section 18.)

7. Limitations

- Sign-up is simulated: no emails are collected. Products are three sample items, not a catalogue.
- The security headers only apply on Netlify. GitHub Pages ignores them.
- Pixel-level match to the Figma design was not measured. Real-device and non-Chrome results are listed above as TO FILL.

8. Next steps

1. Deploy, set the real URL, run Lighthouse, axe and the screen-reader pass; fill the TO FILL cells.
2. Make asset paths root-relative so the 404 page works for any mistyped address.
3. Connect the sign-up to a real email service and the products to a data source.
4. Add checkout, product detail pages, and analytics with a cookie notice.